

Q1. When we say a problem is decidable? Give an example of an undecidable problem. (3 M)

Explain the concept of undecidable problems. (3 M)

What is decidability? How to prove that the given language is undecidable? List some undecidable problems. (7 M)

1. Decidable Problem : A problem that is always solvable is called as decidable problem.

- A problem is said to be decidable if we can always construct an algorithm to solve the problem.
- A language is decidable if there is a turing machine that accepts the language and halts on every input string with an answer yes/no.
- Decidable problems are also known as turing decidable problems because a Turing Machine always halts on every input by accepting or rejecting.

2. Undecidable Problem : A problem that is unsolvable is called as undecidable problem.

- For some inputs, it may produces the correct output, but for all the inputs, it won't produces the correct output.
- There is no algorithm/turing machine that gives an answer for all inputs of a problem.
- In undecidability problem, there will always be a chance that will lead turing machine into an infinite loop without providing answers.
- An **undecidable problem** is one that should give a "yes" or "no" answer, but yet no algorithm exists that can answer correctly on all inputs.

Examples of Undecidable Problems

Example 1:

1. Whether a CFG Generates All Strings or Not

This problem asks:

Does the given Context Free Grammar (CFG) generate **every possible string** over its alphabet?

Example:

If alphabet = {a, b}, then the CFG should generate:

a, b, aa, ab, ba, bb, aaa, ...



Why is it Undecidable?

A CFG can generate **infinitely many strings**.

There is no algorithm that can check every possible string and confirm that **all** strings are generated.

As a Context Free Grammar (CFG) generates infinite strings, we can't ever reach up to the last string and hence it is Undecidable.

Example 2:

Is it possible to convert a given ambiguous CFG into corresponding non-ambiguous CFL ? -

Yes : But, it is also an Undecidable Problem as there doesn't exist any algorithm for the conversion of an ambiguous CFL to non-ambiguous CFL.

Example 3:

Halting Problem

The Halting problem is the most famous of the undecidable problems.

Consider the code

```
num=1;
while(num=0)
{
    num=num+1;
}
```

It counts up forever since it will never equal 0. This is an example of the halting problem.

Example 4: Whether a CFG generates all the strings or not: As a [Context Free Grammar](#) (CFG) generates infinite strings, we can't ever reach up to the last string and hence it is Undecidable.

Example 5: Whether two CFG L and M equal: Since we cannot determine all the strings of any CFG, we can predict that two CFG are equal or not.

Example 6: Ambiguity of CFG: There exist no algorithm which can check whether for the ambiguity of a Context Free Language (CFL). We can only check if any particular string of the CFL generates two different parse trees then the CFL is ambiguous.

How to prove that the given language is undecidable?

We already know: HALT_{TM} is undecidable.

Now we want to prove another language L is undecidable.

Step 1: Start with a Known Impossible Problem

Use:

- Halting Problem
- Acceptance Problem

because they are already undecidable.

Step 2: Assume New Problem Has Solution

Suppose a Turing Machine exists for the new language.

Meaning: it can always answer YES/NO correctly.

Step 3: Use That Machine to Solve Halting Problem

Now connect both problems.

Show: If new problem machine exists, then Halting Problem can also be solved.

Step 4: Contradiction Happens

But we already know: Halting Problem cannot be solved.

So our assumption was wrong.

Therefore: New language is undecidable.

A language is proved undecidable by reducing a known undecidable problem to it. If the new problem could be solved, then the already known undecidable problem would also be solved, which is impossible. Hence the language is undecidable.

Q 2. Mention the difference between decidable and undecidable problems. (4 M)

Aspect	Decidable Problems	Undecidable Problems
Definition	Problems that can be solved by an algorithm that always gives a correct answer in a finite time.	Problems where no algorithm can give a solution for all possible cases.
Solvability	Always solvable using a step-by-step process (algorithm).	Cannot be solved for all inputs using a single algorithm.
Algorithm	There is an algorithm that works for every input and always finishes with an answer.	No algorithm can solve the problem for every input.
Halting	The algorithm stops (halts) and gives an answer for every input.	The algorithm might never stop for some inputs, or no algorithm exists.
Examples	Problems like checking if a number is even or odd, or if a string belongs to a regular language (like finding a match in a search).	Examples include the Halting Problem, where you can't always tell if a program will finish running or run forever.
Decision Procedure	There's a clear method to always reach a correct conclusion.	No guaranteed method exists to solve the problem in every case.
Complexity	May be complex but can always be computed.	Too complex to compute in general, and no universal solution exists.
Applications	Useful in practical computing tasks like compiling code or searching for text patterns.	Helps understand the limits of what computers can do, showing what problems are beyond computation.

Q 3. Regular languages and CFLs are both decidable and Turing-recognizable. Explain whether true or false. (4M)

1. Regular Languages

Regular languages are accepted by: Finite Automata (FA)

Since FA always stops after reading the input:

- every regular language is **decidable**.

Also:

- if a language is decidable, then it is automatically **Turing-recognizable**.

So, regular languages are:

- Decidable ✓
- Turing-recognizable ✓

2. Context Free Languages (CFLs)

CFLs are accepted by: Pushdown Automata (PDA)

There exists algorithms (like parsing algorithms) that can determine whether a string belongs to a CFL and always halt.

Therefore: CFLs are also **decidable**.

And every decidable language is: Turing-recognizable.

So CFLs are:

- Decidable ✓
- Turing-recognizable ✓

Q 4. Explain the difference between decidability and acceptability of a language with respect to TM? (3M)

Decidability	Acceptability
A language is decidable if a Turing Machine always halts and gives answer for every input.	A language is acceptable if a Turing Machine accepts strings in the language, but may run forever for some strings not in the language.
TM always stops.	TM may not stop for some inputs.
For invalid strings, TM rejects and halts.	For invalid strings, TM may reject or loop forever.
Also called Recursive Language .	Also called Recursively Enumerable Language .
More powerful and reliable.	Less strict than decidable languages.

Note :

- **Regular Languages** are accepted by **Finite Automata**.
- **Context Sensitive Languages** are accepted by **Linear Bounded Automata**.
- **Context Free Languages** are accepted by **Push Down Automata**.
- **Recursive Enumerable Languages** are accepted by **Turing Machine**.

Q 5. Recursive Languages and Recursively Enumerable Languages (3 M, 4 M)

When can the language be called a recursive language or a recursively enumerable language? (3 M)

1. Recursive Language

A language is called **recursive** if there exists a Turing Machine that:

- accepts strings belonging to the language, and
- rejects strings not belonging to the language,
- and **always halts** for every input.

The TM always gives a final answer:

- YES (accept)
- or NO (reject)

It never runs forever.

Condition for Recursive Language

A language LLL is recursive if: TM halts on every input string.

Example

- Regular Languages
- Context Free Languages

are recursive languages because algorithms exist that always stop.

2. Recursively Enumerable (RE) Language

A language is called **recursively enumerable** if there exists a Turing Machine that:

- accepts strings belonging to the language,
- but for strings not in the language, it may:
 - reject, or
 - run forever.

TM guarantees answer only for valid strings. For invalid strings, it may never stop.

Condition for RE Language

A language L is recursively enumerable if:

- TM accepts every string in L, but may not halt for strings not in L.

Difference Between Recursive and RE Languages

Recursive Language	Recursively Enumerable Language
TM always halts	TM may run forever
Accepts and rejects correctly	Accepts correctly, but may not reject
Decidable	Turing-recognizable
Every recursive language is RE	Not every RE language is recursive
2 states: accept & halt, reject & halt	3 states: accept & halt, reject & halt, loop or never halt
A language is recursive if TM always halts and gives answer for every input.	A language is recursively enumerable if TM accepts valid strings, but may loop forever for invalid strings.

Important Relation

Recursive Languages $\subseteq$ RE Languages

Meaning:

- Every recursive language is recursively enumerable.
- But every recursively enumerable language is not recursive.

Q 6. Class P, Class NP (4 Marks)

In the world of computers and math, there's this puzzling question: Can every problem we quickly check be solved quickly too? We have two categories: P for problems with quick solutions, and NP for problems where checking is fast, but solving might not be.

Class P contains decision problems that can be solved by a deterministic computer (or deterministic Turing Machine) in **polynomial time**. Problems whose solutions can be found quickly by a computer.

Polynomial time means the running time is: $O(n)$, $O(n^2)$, $O(n^3)$ etc.

These problems are considered: fast, efficient, and easy to solve

Examples of P Problems

- Linear Search
- Sorting numbers
- Searching an element
- Finding shortest path
- Checking whether a number is even

Class NP contains decision problems whose solutions can be: verified quickly in polynomial time, even if finding the solution may be difficult.

NP stands for Non-deterministic Polynomial time

Nondeterministic polynomial-time problems, commonly known as NP problems. These problems have the special property that, once a potential solution is provided, its correctness can be verified quickly. However, finding the solution itself may be computationally difficult.

A well-known example of NP problems is [prime factorization](#). We can verify a factor of an integer in polynomial time. However, we don't know any polynomial time algorithm to factorize a given integer.

Examples of NP Problems

- Travelling Salesman Problem
- Sudoku checking

Relationship Between P and NP

$$P \subseteq NP$$

Meaning:

- Every P problem is also NP.
- But it is still unknown whether: $P=NP$
This is one of the biggest unsolved problems in computer science.

Q 7. Difference between P vs NP problems (4 M)

Feature	P Problems	NP Problems
Solvability	Efficiently solvable in polynomial time.	Efficient verification, solution may not be found efficiently.
Time Complexity	Polynomial time algorithms are known.	Efficient verification algorithms are known, but efficient solution algorithms are not guaranteed.
Nature of Solutions	Solutions can be found efficiently.	Solutions, once proposed, can be verified efficiently.
Known Relationship	P is a subset of NP.	It is unknown whether NP is a proper subset of P or if they are equal.
Practical Implications	Problems in P are considered efficiently solvable in practice.	NP problems are considered computationally hard; no efficient algorithm is currently known, making them potentially impractical for large instances.
Example	Sorting, searching, shortest path problems.	Traveling Salesman , Boolean Satisfiability .

Q 8. NP-hard class / NP-hard problems: No known polynomial-time algorithm exists. NP-Hard does NOT mean - "cannot be solved in polynomial time".

Because:

- maybe in future someone finds a fast algorithm.
- we still do not know whether $P=NP$.

Q 9. NP-complete class / NP complete problems (7 Marks)

A problem is NP-complete if it is both NP and NP-hard. NP-complete problems are the hard problems in NP.

If one could solve an NP-complete problem in polynomial time, then one could also solve any NP problem in polynomial time.

NP-Complete problems are the hardest problems in class NP.

A problem is called **NP-Complete** if:

1. The problem belongs to NP.
2. Every NP problem can be reduced to it in polynomial time.

Simple Meaning

- Finding the solution is difficult.
- Checking the solution is easy.
- If one NP-Complete problem is solved efficiently, then all NP problems can also be solved efficiently.

Conditions for NP-Complete

A problem is NP-Complete when:

- it is in NP, and
- it is NP-Hard.

Formula : $NP\text{-Complete} = NP + NP\text{-Hard}$

Characteristics of NP-Complete Problems

- Hardest problems in NP
- No known polynomial-time algorithm
- Solution verification is easy
- Important in optimization and computational theory

Example: **SAT Problem (Boolean Satisfiability Problem (SAT))**

SAT means: Satisfiability Problem

It asks: "Can we give TRUE or FALSE values to variables so that the whole expression becomes TRUE?"

Example : Consider: $(A+B)(A^{-}+C)$

Here:

- + means OR
- multiplication means AND
- A^{-} means NOT A

So the question is: Can we assign TRUE/FALSE values to A, B, and C to make the whole expression TRUE?

Try Values

Take: A = TRUE, B = FALSE, C = TRUE

Now substitute:

First Bracket

$$(A + B)$$
$$(TRUE + FALSE) = TRUE$$

Second Bracket

$$(\overline{A} + C)$$

Since $A = TRUE$,

$$\overline{A} = FALSE$$

So:

$$(FALSE + TRUE) = TRUE$$

Final Expression

$$(TRUE)(TRUE) = TRUE$$

So the expression becomes TRUE.

Hence:

The SAT problem answer is YES.



because satisfying values exist.

Why is SAT NP-Complete?

Hard Part : Finding correct TRUE/FALSE values may take many trials.

Easy Part

Once someone gives values, checking them is easy and fast.
That is why SAT belongs to NP, and SAT is also NP-Hard.
Therefore: SAT is NP-Complete.

Q 10. Discuss Post's Correspondence Problem with example. (7 Marks)

It answers that Can we select some strings in the same order from both lists so that the final strings become equal?

Post Correspondence Problem is a popular undecidable problem that was introduced by Emil Leon Post in 1946. It is simpler than Halting Problem.

In this problem we have N number of **Dominos** (tiles).

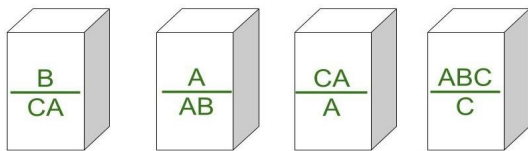
The aim is to arrange tiles in such order that string made by Numerators is same as string made by Denominators.

In simple words, lets assume we have two lists both containing N words, aim is to find out concatenation of these words in some sequence such that both lists yield same result. Let's try understanding this by taking two **lists A and B**

A=[aa, bb, abb] and B=[aab, ba, b]

Now for sequence 1, 2, 1, 3 first list will yield aabbbaaabb and second list will yield same string aabbbaaabb. So the solution to this PCP becomes 1, 2, 1, 3. Post Correspondence Problems can be represented in two ways:

1. Domino's Form :



2. Table Form :

	Numerator	Denominator
1	B	CA
2	A	AB
3	CA	A
4	ABC	C

Example 1:

	A	B
1	1	111
2	10111	10
3	10	0

Explanation -

- **Step-1:** We will start with tile in which numerator and denominator are starting with same number, so we can start with either 1 or 2. Lets go with **second** tile, string made by numerator- 10111, string made by denominator is 10.
- **Step-2:** We need 1s in denominator to match 1s in numerator so we will go with **first** tile, string made by numerator is 10111 1, string made by denominator is 10 111.
- **Step-3:** There is extra 1 in numerator to match this 1 we will add **first** tile in sequence, string made by numerator is now 10111 1 1, string made by denominator is 10 111 111.
- **Step-4:** Now there is extra 1 in denominator to match it we will add **third** tile, string made by numerator is 10111 1 1 10, string made by denominator is 10 111 111 0.

Final Solution - 2 1 1 3

String made by numerators: 101111110

String made by denominators: 101111110

- As you can see, strings are same.
So, solution exists for this Example 1.

Example 2:

A={ba, abb, bab}

B={bab, bb, abb}

The tiles are:

Tile No. Top (A) Bottom (B)

1	ba	bab
2	abb	bb
3	bab	abb

Goal - Find a sequence of tiles such that: **Top string=Bottom string**

Try Tile 1

Sequence: (1)

Top: ba

Bottom: bab

Bottom has extra **b**.

Try Tile 1, 2

Sequence: (1,2)

Top : ba+abb=baabb

Bottom : bab+bb=babbb

Not equal ✕

Try Tile 1, 3

Sequence: (1,3)

Top : ba+bab=babab

Bottom : bab+abb=bababb

Not equal ✕

Try Tile 3

Sequence: (3)

Top : bab

Bottom : abb

Not equal ✕

Continue Checking : After trying possible combinations, no sequence makes both strings equal.

So, For the given PCP:

$A=\{ba, abb, bab\}$

$B=\{bab, bb, abb\}$

no sequence of tiles produces equal top and bottom strings.

Therefore: The given PCP instance has no solution.

We can try unlimited combinations like one above but none of combination will lead us to solution, thus this problem does not have solution.

Undecidability of Post Correspondence Problem : As theorem says that PCP is undecidable. That is, there is no particular algorithm that determines whether any Post Correspondence System has solution or not.

Why PCP is Undecidable?

There can be:

- infinitely many possible combinations of tiles.

No algorithm can check all combinations for every case.

Hence PCP is undecidable.